When NoSQL
When SQL

OLAP vs OLTP

Horizontal partitioning vs vertical partitioning

Partition key
High cardinality = many unique values
Evenly distributed = 90% traffic one partition is bad = hot partition problem

Sequential vs random values
Partitioning : Sequential keys (like date, auto-increment ID) are often GOOD
Sharding:Sequential keys are often **BAD**

Sharding
Sharding key

Indexing
Index , how it works (B Tree , O(logn)
Good Index , where, order by , join

High cardinality → very important
(Bad cardinality example → gender)
Evenly distributed → good, but less critical than sharding
For indexing, query patterns are so important , not just cardinality

Primary index, secondary index
Primary index , PK , actual B tree data row
Secondary , pointer to PK

Consistency model
Strong, eventual, causal
Per session in mongo
Casual keeps the **order** of **related** the event
Events are "**related**" if they happen in the **same session** (or depend on each other).

Mongo DB sharding

Horizontal scaling

Replica set
fault tolerance + high availability
Read scaling
Only write on primary

Sharding
One shard = 1 replica set
Horizontal partitioning of data

In each shard = data replicated = 1 primary , N secondaries

sharding strategies
- Range-based
- Hashed sharding
- Zones sharding
- Compound sharding

Compare pros and cons

For shard key
-High cardinality?
-evenly write distribution
-Included in most queries

CAP

MongoDB **CP** (in strict mode)
Could be different based on consistency level
Mongo could be **AP** too

Cassandra **AP**

Relational DB **AC**

ACID

Atomicity
Consistency?
Isolation
Durability
